江西省应急管理厅
业务应用系统
建设规范

江西省应急管理厅　　发 布

目 录

1 范围 1

2 规范性引用文件 1

3 术语和定义 1

3.1 JAVA 编程语言 JAVA PROGRAMMING LANGUAGE 1
3.2 JAVASCRIPT 编程语言 JAVASCRIPT PROGRAMMING LANGUAGE 1
3.3 超文本标记语言 HYPER TEXT MARKUP LANGUAGE 1
3.4 层叠样式表 CASCADING STYLE SHEETS 2
3.5 表现层状态转移 REPRESENTATIONAL STATE TRANSFER 2

4 前后端分离模式 2

5 后端开发规范 3

5.1 JAVA 编码规范 3
5.1.1 命名规约 3
5.1.2 格式规约 4
5.1.3 注释规约 4
5.1.4 方法设计 4
5.1.5 类设计 4
5.1.6 控制语句 5
5.1.7 时间处理 5
5.1.8 集合处理 6
5.1.9 并发处理 6
5.1.10 异常处理 7
5.1.11 日志规约 7
5.2 RESTFUL API 设计规范 7
5.2.1 请求设计 7
5.2.2 安全设计 10
5.2.3 API 文档 11

6 前端开发规范 13

6.1 JAVASCRIPT 13
6.1.1 命名规约 13
6.1.2 格式规约 14
6.1.3 注释规约 14
6.1.4 基本类型 15
6.1.5 引用类型 15
6.1.6 控制语句 15
6.2 HTML 15
6.3 CSS 15
6.3 用户界面设计 16

7 安全规范 16

7.1 数据安全 16
7.2 源码安全 16
7.3 网络安全 17
7.3.1 物理平台安全 17
7.3.2 网络设备防护 17
7.4 服务器安全 17

前　言

本规范按照 GB/T1.1-2020 《标准化工作导则 第一部分：标准化文件的结构和起草规则》的规定编写。

本规范由江西省防汛信息中心提出并归口。

本规范主要起草单位：江西省水投江河信息技术有限公司。

引　言

为进一步明确省应急管理厅项目建设工作，科学、规范地指导各单位系统开发与对接，特制定此规范。

本规范是规定了江西省应急管理厅业务应用系统建设规范。

本规范用以指引应急管理厅项目应用建设过程中遵循业务应用系统建设规范。

江西省应急管理厅
业务应用系统建设规范

1 范围

本规范规定了江西省应急管理厅业务应用系统建设规范，明确了常见编程语言的相关开发要求。

本规范适用于指导各类应急管理厅应用系统的开发编码、服务接口设计等工作。

2 规范性引用文件

下列标准对于本规范的应用是必不可少的。凡是注日期的引用文件，仅注明日期的版本适用于本文件。凡是不注明日期的引用文件，其最新版本适用于本文件。

GB/T 32619-2016 政务服务中心信息公开编码规范

GB/T 32617-2016 政务服务中心信息公开数据规范

SL 736-2016 水文数据目录服务规范

3 术语和定义

下列术语和定义适用于本文件。

3.1 Java 编程语言 Java programming language

Java 是一门面向对象编程语言，不仅吸收了 C++语言的各种优点，还摒弃了 C++里难以理解的多继承、指针等概念，因此 Java 语言具有功能强大和简单易用两个特征。Java 语言作为静态面向对象编程语言的代表，极好地实现了面向对象理论，允许程序员以优雅的思维方式进行复杂的编程。

3.2 JavaScript 编程语言 JavaScript programming language

JavaScript（简称“JS”） 是一种具有函数优先的轻量级，解释型或即时编译型的编程语言。虽然它是作为开发 Web 页面的脚本语言而出名，但是它也被用到了很多非浏览器环境中，JavaScript 基于原型编程、多范式的动态脚本语言，并且支持面向对象、命令式和声明式（如函数式编程）风格。

3.3 超文本标记语言 Hyper text markup language

超文本标记语言（简称“HTML”）包括一系列标签，通过这些标签可以将网络上的文档格式统一，

使分散的 Internet 资源连接为一个逻辑整体。HTML 文本是由 HTML 命令组成的描述性文本，HTML 命令可以说明文字，图形、动画、声音、表格、链接等。

3.4 层叠样式表 Cascading style sheets

层叠样式表(简称“CSS”)是一种用来表现 HTML（标准通用标记语言的一个应用）或 XML（标准通用标记语言的一个子集）等文件样式的计算机语言。CSS 不仅可以静态地修饰网页，还可以配合各种脚本语言动态地对网页各元素进行格式化。CSS 能够对网页中元素位置的排版进行像素级精确控制，支持几乎所有的字体字号样式，拥有对网页对象和模型样式编辑的能力。

3.5 表现层状态转移 Representational state transfer

RESTful 是一种网络应用程序的设计风格和开发方式，基于 HTTP，可以使用 XML 格式定义或 JSON 格式定义。RESTful 适用于移动互联网厂商作为业务接口的场景，实现第三方 OTT 调用移动网络资源的功能，动作类型为新增、变更、删除所调用资源。

4 前后端分离模式

前后端分离模式是 Web 应用的一种架构模式。在开发期，前后端工程师可以通过约定好交互接口，实现并行开发；在运行期，前后端分离模式需要对 Web 应用进行分离部署，前后端之间使用 RESTful 风格的 HTTP 请求进行交互。通过将开发团队前后端分离化，让前后端工程师只需要专注于前端或后端的开发工作，使得前后端工程师分别自治，前后端分离的主要优势如下：

a) 前后端解耦

前端服务器使用 Nginx 部署静态资源，负责控制页面引用、跳转和路由；后端同样单独部署应用，提供 API 数据接口来响应前端请求，各司其职。前后端分离后，前后端可选用各自的技术栈和发展路线，只需通过协商好的 API 接口进行交互，同时增加了代码的可维护性。

b) 快速定位问题

当应用出现页面逻辑、跳转错误、浏览器兼容性问题、脚本错误、页面样式等问题，全部由前端工程师来负责；接口数据出错，数据没有提交失败，应答超时等问题，全部由后端工程师负责，双方互不干扰。

c) 前后端服务器水平扩展

前后端分离的架构下，后端服务可设计为完全无状态的服务。出现大并发量的情况，能够很好的进行水平扩展，提高应用并发处理能力。

d）减少后端服务器的并发和负载压力

前后端分离后，除 API 调用请求会抵达后端服务器外，其他所有请求全部由前端 Nginx 服务器承担，减少后端压力。

e）后端接口复用

当一套应用有多个展现端（如：APP 端、网页端等），后端提供的 API 接口能够实现高度复用，适配速度快，只需前端工程师开发相应展现端即可。

f）前端热部署

前端页面单独部署在 Nginx 中，依靠 Nginx 的特性，无需重启应用，实现前端无缝升级。

g）提高开发效率

在约定好数据传输格式后，前后端可以并行开发。前端项目中加入 mock 测试等，模拟后台返回的数据，无需依赖后台开发，提高工作效率，且易于集成。

5 后端开发规范

5.1 Java 编码规范

5.1.1 命名规约

a）禁止拼音缩写，避免阅读者费劲猜测；尽量不用拼音，除非中国式业务词汇没有通用易懂的英文对应；

b）禁止使用非标准的英文缩写；

c）禁用其他编程语言风格的前缀和后缀，且代码中的命名均不能以下划线或美元符号开始，也不能以下划线或美元符号结束。在其它编程语言中使用的特殊前缀或后缀，如 _name， name_， mName， i_name，在 Java 中都不建议使用；

d）类名与接口名使用 UpperCamelCase 风格，遵从驼峰形式。Tcp，Xml 等缩写也遵循驼峰形式，实体类及各类数据 POJO 后缀中存在首字母缩写的，全部使用大写形式；

e）方法名、参数名、成员变量、局部变量使用 lowerCamelCase 风格，遵从驼峰形式；

f）常量命名全大写，单词间用下划线隔开。力求语义表达完整清楚；

g）枚举类名以 Enum 结尾；抽象类使用 Abstract 或 Base 开头；异常类使用 Exception 结尾；测试类以它要测试的类名开始，以 Test 结尾；

h）POJO 类中布尔类型的变量名，不要加 is 前缀，否则部分框架解析会引起序列化错误，因使用形容词或动词的过去分词替代，且 1 表示是，0 表示否；

i）避免成员变量，方法参数，局部变量的重名复写，引起混淆。类的私有成员变量名，不与父类的成员变量重名；方法的参数名/局部变量名，不与类的成员变量重名（getter/setter 例外）；下面错误的地方，Java 在编译时都是合法的，但给阅读者带来极大的障碍。

5.1.2 格式规约

a）使用公司统一的代码格式模板，基于 IDE 自动的格式化。在接手旧项目先进行一次全面格式化，可以避免不同开发者之间，因为格式不统一产生代码合并冲突，或者代码变更日志中因为格式不同引起的变更，掩盖了真正的逻辑变更；

b）IDE 的 text file encoding 设置为 UTF-8; IDE 中文件的换行符使用 Unix 格式，不要使用 Windows 格式。

5.1.3 注释规约

a）代码修改的同时，注释也要进行相应的修改。尤其是参数、返回值、异常、核心逻辑等的修改；

b）类、类的公有成员、方法的注释必须使用 Javadoc 规范，使用/** xxx */格式，不得使用//xxx 方式；

正确的 JavaDoc 格式可以在 IDE 中，查看调用方法时，不进入方法即可悬浮提示方法、参数、返回值的意义，提高阅读效率。

5.1.4 方法设计

正被外部调用的接口，不允许修改方法签名，避免对接口的调用方产生影响。只能新增新接口，并对已过时接口加@Deprecated 注解，并清晰地说明新接口是什么。

5.1.5 类设计

a）所有的子类覆写方法，必须加@Override 注解；

比如有时候子类的覆写方法的拼写有误，或方法签名有误，导致没能真正覆写，加@Override 可以准确判断是否覆写成功。而且，如果在父类中对方法签名进行了修改子类会马上编译报错。另外，也能提醒阅读者这是个覆写方法。最后，建议在 IDE 的 Save Action 中配置自动添加 @Override 注解，如果无意间错误同名覆写了父类方法也能被发现。

b）静态方法不能被子类覆写；

c）POJO 类必须覆写 toString 方法；

便于记录日志，排查问题时调用 POJO 的 toString 方法打印其属性值。否则默认的

Object.toString()只打印类名@数字的无效信息。

d) 使用 IDE 或 lombok 工具生成 toString，hashCode 和 equals 方法；

使用 IDE 生成而不是手写，能保证 toString 有统一的格式，equals 和 hashCode 则避免不正确的 Null 值处理。子类生成 toString() 时，还需要勾选父类的属性。

e) Object 的 equals 方法容易抛空指针异常，应使用常量或确定非空的对象来调用 equals。推荐使用 java.util.Objects#equals（JDK7 引入的工具类）；

f) 除了保持兼容性的情况，总是移除无用属性、方法与参数。

特别是 private 的属性、方法、内部类，private 方法上的参数，一旦无用立刻移除。信任代码版本管理系统。

5.1.6 控制语句

a) if, else, for, do, while 语句必须使用大括号，即使只有单条语句；

b) switch 的规则：在一个 switch 块内，每个 case 要么通过 break/return 等来终止，要么注释说明程序将继续执行到哪一个 case 为止；在一个 switch 块内，都必须包含一个 default 语句并且放在最后，即使它什么代码也没有；

c) 所有包装类对象之间值的比较，全部使用 equals 方法比较；

d) BigDecimal 需要使用 compareTo()；

e) Atomic* 系列，不能使用 equals 方法；

f) double 及 float 的比较，要特殊处理；

g) 数字运算表达式，因为先进行等式右边的运算，再赋值给等式左边的变量，所以等式两边的类型要一致；

h) 数字取模的结果不一定是正数，负数取模的结果仍然负数。

5.1.7 时间处理

a) 日期格式化时，传入 pattern 中表示年份统一使用小写的 y；

b) 在日期格式中分清楚大写的 M 和小写的 m，大写的 H 和小写的 h 分别指代的意义；

c) 获取当前毫秒数：System.currentTimeMillis(); 而不是 new Date().getTime()。如果想获取更加精确的纳秒级时间值，使用 System.nanoTime 的方式。在 JDK8 中，针对统计时间等场景，推荐使用 Instant 类；

d) 不允许在程序任何地方中使用：

1）java.sql.Date ；

2）java.sql.Time ；

3）java.sql.Timestamp。

e）不要在程序中写死一年为 365 天，避免在公历闰年时出现日期转换错误或程序逻辑错误。

5.1.8 集合处理

a）不要在 foreach 循环里进行元素的 remove/add 操作，remove 元素可使用 Iterator 方式；

b） 使用 entrySet 遍历 Map 类集合 Key/Value，而不是 keySet 方式进行遍历。keySet 遍历的方式，增加了 N 次用 key 获取 value 的查询；

c）当对象用于集合时，下列情况需要重新实现 hashCode()和 equals()：以对象做为 Map 的 KEY 时；将对象存入 Set 时；

d）长生命周期的集合，里面内容需要及时清理，避免内存泄漏；

e）集合如果存在并发修改的场景，需要使用线程安全的版本。

5.1.9 并发处理

a）创建线程或线程池时请指定有意义的线程名称，方便出错时回溯；

b）线程池不允许使用 Executors 去创建，避资源耗尽风险；

c）正确停止线程；

Thread.stop()不推荐使用，强行的退出太不安全，会导致逻辑不完整，操作不原子，已被定义 Deprecate 方法；

停止单条线程，执行 Thread.interrupt()。

停止线程池：ExecutorService.shutdown()：不允许提交新任务，等待当前任务及队列中的任务全部执行完毕后退出；ExecutorService.shutdownNow()：通过 Thread.interrupt()试图停止所有正在执行的线程，并不再处理还在队列中等待的任务。

最优雅的退出方式是先执行 shutdown()，再执行 shutdownNow()。

注意，Thread.interrupt()并不保证能中断正在运行的线程，需编写可中断退出的 Runnable。

d）编写可停止的 Runnable；

执行 Thread.interrupt() 时，如果线程处于 sleep(), wait(), join(), lock.lockInterruptibly()等 blocking 状态，会抛出 InterruptedException，如果线程未处于上述状态，则将线程状态设为 interrupted。

e) Runnable 中必须捕获一切异常；

f) 全局的非线程安全的对象可考虑使用 ThreadLocal 存放。全局变量包括单例对象，static 成员变量。

5.1.10 异常处理

a) 创建异常的消耗大，只用在真正异常的场景；

构造异常时，需要获得整个调用栈，有一定消耗。不要用来做流程控制，条件控制，因为异常的处理效率比条件判断低。发生概率较高的条件，应该先进行检查规避。

b) 捕获异常一定要处理；如果故意捕获并忽略异常，须要注释写明原因方便后面的阅读者知道，此处不是漏了处理；

c) 异常处理不能吞掉原异常，要么在日志打印，要么在重新抛出的异常里包含原异常；

d) 如果不想处理异常，可以不进行捕获。但最外层的业务使用者，必须处理异常，将其转化为用户可以理解的内容；

e) 必须对资源对象、流对象进行关闭，或使用语法 try-with-resource；

f) 关闭动作必需放在 finally 块，不能放在 try 块 或 catch 块，这是经典的错误；

g) 更加推荐直接使用 JDK7 的 try-with-resource 语法自动关闭 Closeable 的资源，无需在 finally 块处理，避免潜在问题；

h) 如果处理过程中有抛出异常的可能，也要做 try-catch，否则 finally 块中抛出的异常，将代替 try 块中抛出的异常；

i) 不能在 finally 块中使用 return，finally 块中的 return 将代替 try 块中的 return 及 throw Exception。

5.1.11 日志规约

a) 应用中不可直接使用日志库（Log4j、Logback）中的 API，而应使用日志框架 SLF4J 中的 API。使用门面模式的日志框架,有利于维护各个类的日志处理方式统一,亦可使用 Lombok 的@Slf4j 注解；

b) 禁止使用性能很低的 System.out()打印日志信息。同理也禁止 e.printStackTrace(); 例外: 应用启动和关闭时，担心日志框架还未初始化或已关闭。

5.2 RESTful API 设计规范

5.2.1 请求设计

a) 根 URL 设计

API 的根入口点应尽可能保持足够简单，如果应用很庞大或者预计它将会变的很庞大，那应该将 API 放到子域下，这种做法可以保持某些规模化上的灵活性。

b) 版本控制

所有的 API 必须保持向后兼容，你必须在引入新版本 API 的同时确保旧版本 API 仍然可用。所以应该为其提供版本支持。API 资源不同的版本，可以理解成同一种资源的不同表现形式，所以应该采用同一个 URI。

对于一个 API 或服务，应在生产中最多保留 3 个最详细的版本。版本号，分为整型和浮点型：整型： 大功能版本， 如 v1、v2、v3...，浮点型： 补充功能版本， 如 v1.1、v1.2、v2.1、v2.2 ...

以下推荐两种处理方式：

1）版本号约定在 HTTP 请求头信息的 Accept 字段中进行区分；

2） 版本号约定在请求路径中。

c) 资源端点

端点就是指向特定资源或资源集合的 URL。在端点的设计中，开发人员必须遵守下列约定：

1) URL 的命名必须全部小写；

2) URL 中资源（resource）的命名必须是名词，不允许出现动词，并且必须是复数形式；

3) 必须优先使用 Restful 类型的 URL；

4) URL 必须是易读的，使用分隔符/必须用来指定层级关系，避免滥用造成层级不清晰，且 URL 结尾不允许出现分隔符/；

5) URL 一定不可暴露服务器架构及数据库结构；

6) URL 多个单词之间的连接，必须使用连字符-进行分隔，使用下划线可能显示时会被遮蔽及混淆；

7) URL 数量尽量少，接口数量越少越好，能合并的接口就尽量合并。

d) HTTP 动词

对于资源的具体操作类型，由 HTTP 动词表示。常用的 HTTP 动词有下面五个（括号里是对应的 SQL 命令）。

1) GET（SELECT）：从服务器取出资源（一项或多项）；

2) POST（CREATE）：在服务器新建一个资源；

3) PUT（UPDATE）：在服务器更新资源（客户端提供改变后的完整资源）；

4) PATCH（UPDATE）：在服务器更新资源（客户端提供改变的属性）；

5) DELETE（DELETE）：从服务器删除资源。

特殊情况：若项目处于特殊网络环境或安全限制下，可仅使用 GET、POST，其中 POST 请求使用 header 或参数形式注明动作。

针对每一个端点来说，下面列出所有可行的 HTTP 动词和端点的组合：

请求方法	URL	描述
GET	/rivers	列出所有的河流
POST	/rivers	新增一个新的河流
GET	/rivers/{river}	获取指定河流详情
PUT	/rivers/{river}	更新指定河流
PATCH	/rivers/{river}	更新河流(部分对象)
DELETE	/rivers/{river}	删除指定河流
GET	/rivers/{river}/reaches	检索指定河流下的河段列表
GET	/reaches	列出所有河段。
POST	/reaches	新增新的河段
GET	/reaches/{reach}	获取指定的河段详情
PUT	/reaches/{reach}	更新指定的河段
PATCH	/reaches/{reach}	更新指定的河段(部分对象)
GET	/reach-types	获取所有河段类型
GET	/reach-types/{type}	获取指定的河段类型详情

请求方法	URL	描述
GET	/employees	检索整个巡河员列表
GET	/employees/{employee}	检索指定特定的巡河员
GET	/rivers/{river}/employees	检索在这个河流工作的巡河员的名单
POST	/employees	新增指定新巡河员
POST	/rivers/{river}/employees	在特定的河流雇佣一名巡河员
DELETE	/rivers/{river}/employees/{employee}	从某个河流解雇一名巡河员

5.2.2 安全设计

a）协议

HTTP 协议是以明文方式发送内容，不提供任何方式的数据加密，如果攻击者截取了客户端和服务器之间的传输报文，就可以直接读懂其中的信息，因此 HTTP 协议不适合传输一些敏感信息，比如信用卡号、密码等。安全套接字层超文本传输协议（HTTPS），为了数据传输的安全，HTTPS 在 HTTP 的基础上加入了 SSL 协议，SSL 依靠证书来验证服务器的身份，并为客户端和服务器之间的通信加密。对于安全性要求较高的应用，应当全部使用 HTTPS 协议。

b）限流

对外提供的 API 接口设计都要考虑限流，如果不考虑限流，会成系统的连锁反应，轻者响应缓慢，重者系统宕机，整个业务线崩溃。系统在设计之初就会有一个预估容量，长时间超过系统能承受的 TPS/QPS 阈值，系统可能会被压垮，最终导致整个服务不够用。为了避免这种情况，我们就需要对接口请求进行限流。 限流的目的是通过对并发访问请求进行限速或者一个时间窗口内的的请求数量进行限速来保护系统，一旦达到限制速率则可以拒绝服务、排队或等待。

c）鉴权

1） Token 授权

用户使用用户名密码登录后服务器给客户端返回一个 Token（例如 JWT Token），并将 Token-UserId 以键值对的形式存放在缓存服务器中。服务端接收到请求后进行 Token 验证，如果 Token 不存在，说明请求无效。Token 是客户端访问服务端的凭证。

2) 超时机制

用户每次请求都带上当前时间的时间戳 timestamp，服务端接收到 timestamp 后跟当前时间进行比对，如果时间差大于一定时间（比如 5 分钟），则认为该请求失效。时间戳超时机制是防御 DOS 攻击的有效手段。

3) 签名机制

将 Token 和时间戳加上其他请求参数再用算法加密，加密后的数据就是本次请求的签名 sign，服务端接收到请求后以同样的算法得到签名，并跟当前的签名进行比对，如果不一样，说明参数被更改过，直接返回错误标识。签名机制保证了数据不会被篡改。

4) 防重放

客户端第一次访问时，将签名 sign 存放到缓存服务器中，超时时间设定为跟时间戳的超时时间一致，二者时间一致可以保证无论在 timestamp 限定时间内还是外 URL 都只能访问一次。如果有人使用同一个 URL 再次访问，如果发现缓存服务器中已经存在了本次签名，则拒绝服务。如果在缓存中的签名失效的情况下，有人使用同一个 URL 再次访问，则会被时间戳超时机制拦截。这就是为什么要求时间戳的超时时间要设定为跟时间戳的超时时间一致。拒绝重复调用机制确保 URL 被别人截获了也无法使用（如抓取数据）。

d) 审计日志

使用拦截器、AOP 等技术结合异步日志服务，实现对请求日志的记录与分析，以便进行必要的安全审计及访问轨迹留存。

5.2.3 API 文档

a) 文档规范

尽量采用自动化接口文档，可以做到在线测试，同步更新。

文档内容应包含：接口 BASE 地址、接口版本、接口模块分类等。

每个接口应包含：

1) 接口地址：不包含接口 BASE 地址。

2) 请求方式: get、post、put、delete 等。

3) 请求参数：数据格式【默认 JSON、可选 form data】、

4) 数据类型、是否必填、中文描述。

5) 响应参数：类型、中文描述。

b) 文档模板

1）接口基本信息

接口名称	查询日常监督服务		
接口功能描述	根据主键 id 查询日常监督问题。		
接口地址	/v1/daily/inspect/question/{id}		
技术协议	RESTful	**请求方法**	GET

2）请求头设置

名称	**说明**	**数据类型**	**值**
Content-Type	内容类型	String	application/json
Authorization	授权头		Bearer +空格+token 令牌

3）请求参数

参数名	**必选**	**类型**	**数据限制**	**说明**
id	是	Integer	最大位数 11 位	问题标识 id

4）返回参数

参数名	**类型**	**说明**
id	Integer	主键 ID
mark	String	备注
status	String	状态：0 未下发，1 已下发

5）返回报文样例

```
{
    "code": 200,
    "data": {
```

```
        "id": 34,
        "mark": "备注",
        "status": "0"
    },
    "message": "SUCCESS"
}
```

6）变更记录

操作	负责人	操作时间	变更内容说明
新增	xxx	2020-04-09	新增接口
修改	xxx	2020-04-12	修改格式

6 前端开发规范

6.1 JavaScript

6.1.1 命名规约

a）变量命名使用小驼峰命名规则，命名规范：类型+对象描述的方式，如果没有明确的类型，就可以使前缀为名词。不可以使用动词作为前缀；

b）函数命名使用小驼峰命名规则（构造函数和装饰器使用大驼峰命名）,命名规范：动词作为前缀（构造函数与构造器除外），描述此函数的功能；

can 判断是否可执行某个动作 （ 权限 ） 函数返回一个布尔值。true：可执行；false：不可执行；

has 判断是否含有某个值　函数返回一个布尔值。true：含有此值；false：不含有此值；

is　判断是否为某个值　　函数返回一个布尔值。true：为某个值；false：不为某个值；

get 获取某个值　函数返回一个非布尔值；

set 设置某个值　无返回值、返回是否设置成功或者返回链式对象；

c）类和接口的取名必须使用名词，尽量见名知意，严禁使用拼音和英文混搭；

d）类和接口的取名使用大驼峰命名

e）常量和枚举等静态值使用全大写命名【TS only】；

f）变量声明时必须指定类型，不允许使用 any ，或者声明变量的同时赋值使用类型推导。【TS only】。

6.1.2 格式规约

a）使用两个空格进行缩进；

b）关键字后面加空格；

c）字符串拼接操作符之间要留空格；

d）object 对象等定义时冒号后面加空格,冒号前无空格；

e）逗号后面加空格；

f）for 循环中分号后面加空格；

g）else 关键字要与大括号保持在同一行；

h）if 语句任何情况下大括号不能省略；

i）在允许长度范围内，三元运算符保持在一行，且以空格隔开；

j）每个变量声明都单独分开；

k）多行字符串拼接加号位于行末或者使用模板字符串表达；

l）点号操作符须与属性需在同一行,非特殊情况，避免换行出现；

m）不要省去小数点前面的 0。

6.1.3 注释规约

a）单行注释规约；

1）双斜线与注释文字之间保留一个空格；

2）在代码后面添加注释：//(双斜线)与代码之间保留一个空格，并且//(双斜线)与注释文字之间保留一个空格；

3）注释代码：//(双斜线)与代码之间保留一个空格。

b）多行注释规约

1） 若开始(/*和结束(*/)都在一行，推荐采用单行注释；

2） 若至少三行注释时，第一行为/*，最后行为*/，其他行以*开始，并且注释文字与*保留一个空格。

c）函数(方法)注释也是多行注释的一种，但是包含了特殊的注释要求；

d) 类、类的公有成员、方法的注释必须使用 Javadoc 规范，使用/** xxx */格式，不得使用//xxx 方式（同上文 java 注释规约）。

6.1.4 基本类型

a) 使用 let const 替代 var 申明变量；

b) 在同一块作用域禁止重复声明变量；

c) 除需要转义的情况外，字符串统一使用单引号，模板字符串使用反引号；

d) 编码过程中不要使用隐式装换逻辑，如需要进行转换则应明确转换过程。

6.1.5 引用类型

a) 声明数组时必须指定成员类型。【TS only】；

b) 数组中不允许出现空位；

c) 对象中不要定义重复的 key；

d) 继承父类的子类必需在构造函数中第一行调用 super()；

e) 使用箭头函数的情形下，如果入参只有一个则采用简写模式，如果返回值是一个表达式，采用简写模式；

f) 运行在函数内部的函数。内部函数禁止直接调用外部函数的局部变量，应该使用传参的方式传入，避免内存泄露。

6.1.6 控制语句

switch 一定要使用 break 来将条件分支正常中断。

6.2 HTML

a) 所有页面元素的 z-index 属性值控制在 1000 以下，将 1000-2000 留给组件级调用，2000 以上用于解决特殊情况下的适应问题；

b) 所有的标签及属性名采用小写，DOCTYPE 除外；

c) 自定义标签保证具有语义化。在项目中尽量统一自己的组件的命名规则；

d) 分离思想，所有的样式除特殊情况，css 样式不能内联写在 style 属性中；

e) 行内元素不允许嵌套块元素。

6.3 CSS

a) 选择器 与 { 之间必须包含空格；

b) 多个属性分行书写；

c) 当一个 rule 包含多个 selector 时，每个选择器声明必须独占一行【not scss&less】；

d) 使用子选择器替代后代选择器；

e) 禁止使用!important 进行样式覆盖。如必须使用，必须使用标准注释体并且说明使用原因；

f) 属性定义后必须以分号结尾；

g) 使用 border / margin / padding 等缩写时，应注意隐含值对实际数值的影响，确实需要设置多个方向的值时才使用缩写；

h) 当数值为 0 - 1 之间的小数时，省略整数部分的 0；

i) 颜色值可以缩写时，必须使用缩写形式；

j) 颜色值不允许使用命名色值；

k) 必须同时给出水平和垂直方向的位置；

l) SCSS mixin 的方法参数括号与 { 之间必须包含一个空格，各参数间必须有一个空格【scss only】；

m) 多个并行选择器使用同一规则，必须换行【scss only】；

n) media query 推荐直接在选择器的嵌套中使用，有助于保持媒体查询属于的上下文【scss&less only】。

6.3 用户界面设计

前端在设计用户界面时应当遵循《江西省应急管理厅应用系统用户界面设计规范》。

7 安全规范

7.1 数据安全

a) 开发人员不允许通过后台私自访问正式环境系统数据库获取数据；

b) 对于应用系统的数据调用需先申请，申请审批通过后方能开放相关接口；

c) 系统相关管理人员需做好系统账号安全性管理，并对系统数据库、服务器访问进行严格控制，并做好留痕工作。

7.2 源码安全

a) 自主研发系统相关源代码产权应归系统需求方所有；

b）系统源代码需存放在内网服务器，不允许将系统源代码存放在开源平台等类似互联网无权限控制的服务器或系统中；

c）对源代码的访问需进行权限控制并设置访问期限，授权后方可访问，超出访问期限需收回相关权限。同时要求源代码访问人员签署相关协议，禁止将系统源代码对外售卖或私自分享。

7.3 网络安全

7.3.1 物理平台安全

a）如无需提供互联网服务的系统原则上要求部署在内网；

b）服务器需安装安全软件，并定期对服务器进行扫描和对安全软件进行定期升级。发现系统漏洞需及时修复；

c）面向互联网系统需再架设防火墙设备，互联网请求需经过防火墙方可访问到应用服务器。

7.3.2 网络设备防护

a）口令管理

对登陆网络设备的用户应进行身份鉴别，删除默认用户或修改默认用户的口令，根据管理需要开设用户，不得使用缺省口令、空口令、弱口令。此外，主要网络设备对同一用户应支持多种鉴别技术进行身份鉴别，用户口令需定期进行更新，一年不少于4次。

b）登陆管理

限制网络设备的管理员登陆地址，所有需远程管理和监控的网络设备应开启SNMP V3协议，对暂不支持SNMP V3协议的设备可先开通其V2或V1协议，并将其部署在非核心位置。同时，具有登陆失败处理功能，可采取结束会话、限制非法登陆次数和当网络登陆连续超时自动登出等措施。

7.4 服务器安全

a）服务器账号管理

由于root权限为系统最高权限，可以对系统进行所有管理配置操作，一旦被攻击者获取利用，将严重威胁网站的安全性。建议禁用root直接登录，开放普通用户登录，通过普通用户登录再su进行系统管理。

b）服务器日志保存

根据《网络安全法》、《网络安全等级保护管理条例》规定，服务器日志应保存不少于6个月。

c）存储型跨站脚本攻击处理

跨站脚本攻击，XSS又叫CSS (Cross Site Script)。XSS属于被动式的攻击，它指的是恶意攻击者往Web页面里插入恶意html代码，当用户浏览该页之时，嵌入其中Web里面的html代码会被执行，从而达到恶意用户的特殊目的。攻击者经常利用跨站脚本攻击的方式进行网络钓鱼行为。或诱骗用户点击含有跨站脚本攻击的连接，从而获取用户的COOKIE等信息，绕过用户身份认证，直接进入用户登录页面。

d）服务器信息安全

网站部署完毕不应含有服务器配置、版本、运行环境信息，这些信息可能为攻击者了解网站配置，进行下一步攻击提供帮助，需要将这些敏感文件进行删除、访问权限控制。